# Underactuated Robotics

***Algorithms for Walking, Running, Swimming, Flying, and Manipulation***

Russ Tedrake


Last modified 2024-11-11.
How to cite these notes, use annotations, and give feedback.

**Note:** These are working notes used for a course being taught at MIT. They will be updated throughout the Spring 2024 semester. Lecture videos are available on YouTube.

Previous Chapter | Table of contents | Next Chapter

Launch in Deepnote

# Appendix B
# Multi-Body Dynamics

## B.1 Deriving the Equations of Motion

The equations of motion for a standard robot can be derived using the method of Lagrange. Using $T$ as the total kinetic energy of the system, and $U$ as the total potential energy of the system, $L = T - U$, and $\tau_i$ as the element of the generalized force vector corresponding to $q_i$, the Lagrangian dynamic equations are:

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = \tau_i. \tag{1}$$

You can think of them as a generalization of Newton's equations. For a particle, we have $T = \frac{1}{2}m\dot{x}^2$, so $\frac{d}{dt}\frac{\partial L}{\partial \dot{x}} = m\ddot{x}$, and $\frac{\partial L}{\partial x} = -\frac{\partial U}{\partial x} = f$ amounting to $f = ma$. But the Lagrangian derivation works in generalized coordinate systems and for constrained motion.

**Example B.1 (Simple Double Pendulum)**

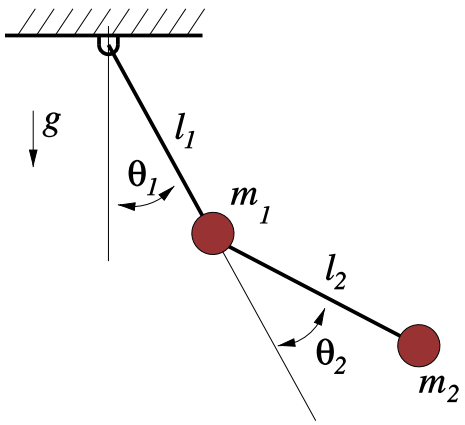



Figure B.1 - Simple double pendulum

Consider the simple double pendulum with torque actuation at both joints and all of the mass concentrated in two points (for simplicity). Using $\mathbf{q} = [\theta_1, \theta_2]^T$, and $\mathbf{p}_1, \mathbf{p}_2$ to denote the locations of $m_1, m_2$, respectively, the kinematics of this system are:

$$\begin{aligned}\mathbf{p}_1 =& l_1 \begin{bmatrix} s_1 \\ -c_1 \end{bmatrix}, \mathbf{p}_2 = \mathbf{p}_1 + l_2 \begin{bmatrix} s_{1+2} \\ -c_{1+2} \end{bmatrix} \\ \dot{\mathbf{p}}_1 =& l_1 \dot{q}_1 \begin{bmatrix} c_1 \\ s_1 \end{bmatrix}, \dot{\mathbf{p}}_2 = \dot{\mathbf{p}}_1 + l_2 (\dot{q}_1 + \dot{q}_2) \begin{bmatrix} c_{1+2} \\ s_{1+2} \end{bmatrix}\end{aligned}$$

Note that $s_1$ is shorthand for $\sin(q_1)$, $c_{1+2}$ is shorthand for $\cos(q_1 + q_2)$, etc. From this we can write the kinetic and potential energy:

$$\begin{aligned}T =& \frac{1}{2} m_1 \dot{\mathbf{p}}_1^T \dot{\mathbf{p}}_1 + \frac{1}{2} m_2 \dot{\mathbf{p}}_2^T \dot{\mathbf{p}}_2 \\ =& \frac{1}{2}(m_1 + m_2) l_1^2 \dot{q}_1^2 + \frac{1}{2} m_2 l_2^2 (\dot{q}_1 + \dot{q}_2)^2 + m_2 l_1 l_2 \dot{q}_1 (\dot{q}_1 + \dot{q}_2) c_2 \\ U =& m_1 g y_1 + m_2 g y_2 = -(m_1 + m_2) g l_1 c_1 - m_2 g l_2 c_{1+2}\end{aligned}$$

Taking the partial derivatives

$$\begin{aligned}\frac{\partial T}{\partial q_1} &= 0, \\ \frac{\partial U}{\partial q_1} &= (m_1 + m_2) g l_1 s_1 + m_2 g l_2 s_{1+2}, \\ \frac{\partial T}{\partial q_2} &= -m_2 l_1 l_2 \dot{q}_1 (\dot{q}_1 + \dot{q}_2) s_2, \\ \frac{\partial U}{\partial q_2} &= m_2 g l_2 s_{1+2}, \\ \frac{\partial T}{\partial \dot{q}_1} &= (m_1 + m_2) l_1^2 \dot{q}_1 + m_2 l_2^2 (\dot{q}_1 + \dot{q}_2) + m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) c_2, \\ \frac{\partial T}{\partial \dot{q}_2} &= m_2 l_2^2 (\dot{q}_1 + \dot{q}_2) + m_2 l_1 l_2 \dot{q}_1 c_2, \\ \frac{d}{dt}\frac{\partial T}{\partial \dot{q}_1} &= (m_1 + m_2) l_1^2 \ddot{q}_1 + m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 (2\ddot{q}_1 + \ddot{q}_2) c_2 - m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2 \\ \frac{d}{dt}\frac{\partial T}{\partial \dot{q}_2} &= m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 \ddot{q}_1 c_2 - m_2 l_1 l_2 \dot{q}_1 \dot{q}_2 s_2,\end{aligned}$$

and plugging them into the Lagrangian, reveals the equations of motion:

$$\begin{aligned}(m_1 + m_2) l_1^2 \ddot{q}_1 + m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 (2\ddot{q}_1 + \ddot{q}_2) c_2 \\ - m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) \dot{q}_2 s_2 + (m_1 + m_2) l_1 g s_1 + m_2 g l_2 s_{1+2} = \tau_1 \\ m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 \ddot{q}_1 c_2 + m_2 l_1 l_2 \dot{q}_1^2 s_2 + m_2 g l_2 s_{1+2} = \tau_2\end{aligned}$$

As we saw in chapter 1, numerically integrating (and animating) these equations in DRAKE produces the expected result.

If you are not comfortable with these equations, then any good book chapter on rigid body mechanics can bring you up to speed. [1] is an excellent practical guide to robot kinematics/dynamics. But [2] is my favorite mechanics book, by far; I highly recommend it if you want something more. It's also ok to continue on for now thinking of Lagrangian mechanics simply as a recipe than you can apply in a great many situations to generate the equations of motion.

I've also included a few more details about the connections between these equations and the variational principles of mechanics in the notes below, to the extent they may prove useful in our quest to write better optimizations for mechanical systems.

### B.2 The Manipulator Equations

If you crank through the Lagrangian dynamics for a few simple robotic manipulators, you will begin to see a pattern emerge - the resulting equations of motion all have a characteristic form. For example, the kinetic energy of your robot can always be written in the form:

$$T = \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{M}(\mathbf{q})\dot{\mathbf{q}}, \tag{2}$$

where $\mathbf{M}$ is the state-dependent inertia matrix (aka mass matrix). This observation affords some insight into general manipulator dynamics - for example we know that $\mathbf{M}$ is always positive definite, and symmetric[3, p.107] and has a beautiful sparsity pattern[4] that we should take advantage of in our algorithms.

Continuing our abstractions, we find that the equations of motion of a general robotic manipulator (without kinematic loops) take the form

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u}, \tag{3}$$

where $\mathbf{q}$ is the joint position vector, $\mathbf{M}$ is the inertia matrix, $\mathbf{C}$ captures Coriolis forces, and $\tau_g$ is the gravity vector. The matrix $\mathbf{B}$ maps control inputs $\mathbf{u}$ into generalized forces. Note that we pair $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}}$ together on the left side because these terms together represent the force of inertia; the contribution from kinetic energy to the Lagrangian:

$$\frac{d}{dt}\frac{\partial T}{\partial \dot{\mathbf{q}}}^T - \frac{\partial T}{\partial \mathbf{q}}^T = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}}.$$

Whenever I write Eq. (3), I see $ma = f$.

**Example B.2 (Manipulator Equation form of the Simple Double Pendulum)**
The equations of motion from Example 1 can be written compactly as:

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} (m_1+m_2)l_1^2 + m_2 l_2^2 + 2m_2 l_1 l_2 c_2 & m_2 l_2^2 + m_2 l_1 l_2 c_2 \\ m_2 l_2^2 + m_2 l_1 l_2 c_2 & m_2 l_2^2 \end{bmatrix}$$

$$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} 0 & -m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) s_2 \\ \frac{1}{2} m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) s_2 & -\frac{1}{2} m_2 l_1 l_2 \dot{q}_1 s_2 \end{bmatrix}$$

$$\tau_g(\mathbf{q}) = -g \begin{bmatrix} (m_1+m_2) l_1 s_1 + m_2 l_2 s_{1+2} \\ m_2 l_2 s_{1+2} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Note that this choice of the $\mathbf{C}$ matrix was not unique.

The manipulator equations are very general, but they do define some important characteristics. For example, $\ddot{\mathbf{q}}$ is (state-dependent) linearly related to the control input, $\mathbf{u}$. This observation justifies the control-affine form of the dynamics assumed throughout the notes. There are many other important properties that might prove useful. For instance, we know that the $i$ element of $\mathbf{C}\dot{\mathbf{q}}$ is given by [3, § 5.2.3]:

$$[\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}]_i = \dot{\mathbf{q}}^T \bar{\mathbf{C}}_i \dot{\mathbf{q}}, \qquad \bar{C}_{i;j,k}(\mathbf{q}) = \frac{1}{2}\left[\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i}\right],$$

(e.g. it also quadratic in $\dot{\mathbf{q}}$). For the appropriate choice of $\mathbf{C}$, we have that $\dot{\mathbf{M}}(\mathbf{q}) - 2\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ is skew-symmetric[5, § 9.1].

Note that we have chosen to use the notation of second-order systems (with $\dot{\mathbf{q}}$ and $\ddot{\mathbf{q}}$ appearing in the equations) throughout this book. Although I believe it provides more clarity, there is an important limitation to this notation: it is impossible to describe 3D rotations in "minimal coordinates" using this notation without introducing kinematic singularities (like the famous "gimbal lock"). For instance, a common singularity-free choice for representing a 3D rotation is the unit quaternion, described by 4 real values (plus a norm constraint). However we can still represent the rotational velocity without singularities using just 3 real values. This means that the length of our velocity vector is no longer the same as the length of our position vector. For this reason, you will see that most of the software in DRAKE uses the more general notation with $\mathbf{v}$ to represent velocity, $\mathbf{q}$ to represent positions, and the manipulator equations are written as

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} + \mathbf{C}(\mathbf{q}, \mathbf{v})\mathbf{v} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u}, \tag{4}$$

which is not necessarily a second-order system. $\mathbf{v}$ and $\dot{\mathbf{q}}$ are related linearly by $\dot{\mathbf{q}} = \mathbf{N}(\mathbf{q})\mathbf{v}$. Although $\mathbf{N}(\mathbf{q})$ is not necessarily square, its left Moore-Penrose pseudo-inverse $\mathbf{N}^+(\mathbf{q})$ can be used to invert that relationship without residual error, provided that $\dot{\mathbf{q}}$ is in the range space of $\mathbf{N}(\mathbf{q})$ (that is, if it could have been produced as $\dot{\mathbf{q}} = \mathbf{N}(\mathbf{q})\mathbf{v}$ for some $\mathbf{v}$). See [6] for a nice discussion of this topic.

### B.2.1 Recursive Dynamics Algorithms

The equations of motions for our machines get complicated quickly. Fortunately, for robots with a tree-link kinematic structure, there are very efficient and natural recursive algorithms for generating these equations of motion. For a detailed

reference on these methods, see [7]; some people prefer reading about the Articulated Body Method in [8]. The implementation in DRAKE uses a related formulation from [9].

### B.2.2 Bilateral Position Constraints

If our robot has closed-kinematic chains, for instance those that arise from a four-bar linkage, then we need a little more. The Lagrangian machinery above assumes "minimal coordinates"; if our state vector $\mathbf{q}$ contains all of the links in the kinematic chain, then we do not have a minimal parameterization -- each kinematic loop adds (at least) one constraint so should remove (at least) one degree of freedom. Although some constraints can be solved away, the more general solution is to use the Lagrangian to derive the dynamics of the unconstrained system (a kinematic tree without the closed-loop constraints), then add additional generalized forces that ensure that the constraint is always satisfied.

Consider the constraint equation

$$\mathbf{h}(\mathbf{q}) = 0. \tag{5}$$

For example, in the case of the kinematic closed-chain, this can be the kinematic constraint that the location of one end of the chain is equal to the location of the other end of the chain. In the following derivation, we'll be using the time derivatives of this constraint:

$$\dot{\mathbf{h}} = \mathbf{H}\mathbf{v} = 0, \tag{6}$$
$$\ddot{\mathbf{h}} = \mathbf{H}\dot{\mathbf{v}} + \dot{\mathbf{H}}\mathbf{v} = 0, \tag{7}$$

where $\mathbf{H}(\mathbf{q}) = \frac{\partial \mathbf{h}}{\partial \mathbf{q}}\mathbf{N}(\mathbf{q})$ is the "Jacobian with respect to $\mathbf{v}$". Note that $\dot{\mathbf{H}}\mathbf{v}$ is best calculated directly (rather than trying to solve for $\dot{\mathbf{H}}$ as an object by itself).

The equations of motion can be written

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} + \mathbf{C}(\mathbf{q}, \mathbf{v})\mathbf{v} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u} + \mathbf{H}^T(\mathbf{q})\lambda, \tag{8}$$

where $\lambda$ is the constraint force. Let's use the shorthand

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} = \tau(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{u}) + \mathbf{H}^T(\mathbf{q})\lambda. \tag{9}$$

In an optimization-based formulation (for instance, to implement this constraint in a trajectory optimization), we can simply make $\lambda$ a decision variable, and use equations (7) and (9) as constraints.

In other cases, it can be desirable to solve explicitly for $\lambda$. Inserting the dynamics (9) into (7) yields

$$\lambda = -(\mathbf{H}\mathbf{M}^{-1}\mathbf{H}^T)^+(\mathbf{H}\mathbf{M}^{-1}\tau + \dot{\mathbf{H}}\mathbf{v}). \tag{10}$$

In many cases, this matrix will be full rank (for instance, in the case of multiple independent four-bar linkages) and the traditional inverse could have been used.

When the matrix drops rank (multiple solutions), then the pseudo-inverse will select the solution with the smallest constraint forces in the least-squares sense.

To combat numerical "constraint drift", one might like to add a restoring force in the event that the constraint is not satisfied to numerical precision. To accomplish this, rather than solving for $\ddot{\mathbf{h}} = 0$ as above, we can instead solve for

$$\ddot{\mathbf{h}} = \mathbf{H}\dot{\mathbf{v}} + \dot{\mathbf{H}}\mathbf{v} = -2\alpha\dot{\mathbf{h}} - \alpha^2\mathbf{h}, \tag{11}$$

where $\alpha > 0$ is a stiffness parameter. This is known as Baumgarte's stabilization technique, implemented here with a single parameter to provide a critically-damped response. Carrying this through yields

$$\lambda = -(\mathbf{H}\mathbf{M}^{-1}\mathbf{H}^T)^+(\mathbf{H}\mathbf{M}^{-1}\tau + \dot{\mathbf{H}}\mathbf{v} + 2\alpha\mathbf{H}\mathbf{v} + \alpha^2\mathbf{h}). \tag{12}$$

There is an important optimization-based derivation/interpretation of these equations, which we will build on below, using Gauss's Principle of Least Constraint. This principle says that we can derive the constrained dynamics as :

$$\begin{aligned} \min_{\dot{\mathbf{v}}} \quad & \frac{1}{2}(\dot{\mathbf{v}} - \dot{\mathbf{v}}_{uc})^T\mathbf{M}(\dot{\mathbf{v}} - \dot{\mathbf{v}}_{uc}), \\ \text{subject to} \quad & \ddot{\mathbf{h}}(\mathbf{q}, \mathbf{v}, \dot{\mathbf{v}}) = 0 = \mathbf{H}\dot{\mathbf{v}} + \dot{\mathbf{H}}\mathbf{v}, \end{aligned} \tag{13}$$

where $\dot{\mathbf{v}}_{uc} = \mathbf{M}^{-1}\tau$ is the "unconstrained acceleration" [10]. Equation (8) comes right out of the optimality conditions with $\lambda$ acting precisely as the Lagrange multiplier. This is a convex quadratic program with equality constraints, which has a closed-form solution that is precisely Equation (10) above. It is also illustrative to look at the dual formulation of this optimization, which can be written as

$$\min_{\lambda} \frac{1}{2}\lambda^T\mathbf{H}\mathbf{M}^{-1}\mathbf{H}^T\lambda - \lambda^T(\mathbf{H}\mathbf{M}^{-1}\tau + \dot{\mathbf{H}}\mathbf{v}).$$

Observe that the linear term is contains the acceleration of $\mathbf{h}$ if the dynamics evolved without the constraint forces applied; let's call that $\ddot{\mathbf{h}}_{uc}$:

$$\min_{\lambda} \frac{1}{2}\lambda^T\mathbf{H}\mathbf{M}^{-1}\mathbf{H}^T\lambda - \lambda^T\ddot{\mathbf{h}}_{uc}.$$

The primal formulation has accelerations as the decision variables, and the dual formulation has constraint forces as the decision variables. For now this is merely a cute observation; we'll build on it more when we get to discussing contact forces.

### B.2.3 Bilateral Velocity Constraints

Consider the constraint equation

$$\mathbf{h}_v(\mathbf{q}, \mathbf{v}) = 0, \tag{14}$$

where $\frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \neq 0$. These are less common, but arise when, for instance, a joint is driven through a prescribed motion. Again, we will use the derivatives,

$$\dot{\mathbf{h}}_v = \frac{\partial \mathbf{h}_v}{\partial \mathbf{q}} \mathbf{N}(\mathbf{q})\mathbf{v} + \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \dot{\mathbf{v}} = 0, \tag{15}$$

and, the manipulator equations given by

$$\mathbf{M}\dot{\mathbf{v}} = \tau + \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}}^T \lambda. \tag{16}$$

These constraints can be added directly to an optimization with $\lambda$ as a decision variable. Alternatively, we can solve for $\lambda$ explicitly yielding

$$\lambda = -\left( \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \mathbf{M}^{-1} \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \right)^+ \left[ \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \mathbf{M}^{-1} \tau + \frac{\partial \mathbf{h}_v}{\partial \mathbf{q}} \mathbf{N}(\mathbf{q})\mathbf{v} \right]. \tag{17}$$

Again, to combat constraint drift, we can ask instead for $\dot{\mathbf{h}}_v = -\alpha \mathbf{h}_v$, which yields

$$\lambda = -\left( \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \mathbf{M}^{-1} \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \right)^+ \left[ \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \mathbf{M}^{-1} \tau + \frac{\partial \mathbf{h}_v}{\partial \mathbf{q}} \mathbf{N}(\mathbf{q})\mathbf{v} + \alpha \mathbf{h}_v \right]. \tag{18}$$

**Example B.3 (Joint locking)**

Consider the simple, but important, case of "locking" a joint (making a particular joint act as if it is welded in the current configuration). One could write this as a bilateral velocity constraint

$$\mathbf{h}_v(\mathbf{q}, \mathbf{v}) = v_i = 0.$$

Since $\frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} = \mathbf{e}_i$, (the vector with $1$ in the $i$th element and zero everywhere else, equation (15) reduces to $\dot{v}_i = 0$, and $\frac{\partial \mathbf{h}_v}{\partial \mathbf{v}}^T \lambda$ impacts the equation for $\dot{v}_i$ and \emph{only} the equation for $\dot{v}_i$. The resulting equations can be obtained by simply removing the terms associated with $v_i$ from the manipulator dynamics (and setting $\dot{v}_i = 0$).

### B.2.4 Hybrid models via constraint forces

For hybrid models of contact, it can be useful to work with a description of the dynamics in the floating-base coordinates and solve for the contact forces which enforce the constraints in each mode. We'll define a mode by a list of pairs of contact geometries that are in contact, and even define whether each contact is in sticking contact (with zero relative tangential velocity) or sliding contact. We can ask our geometry engine to return the contact points on the bodies, ${}^A\mathbf{p}^{C_a}$ and ${}^B\mathbf{p}^{C_b}$, where $C_a$ is the contact point (or closest point to contact) in body $A$ and $C_b$ is the contact point (or closest point to contact) in body $B$. More generally, we will refer to $C_a$ and $C_b$ as contact frames with their origins at these points, their $z$-axis pointing out along the contact normal, and their $x-y$-axes chosen as an orthonormal basis orthogonal to the normal.

For each sticking contact, we have a bilateral position constraint of the form

$$\mathbf{h}_{A,B}(\mathbf{q}) = {}^{W}\mathbf{p}_{C_a}^{C_a} - {}^{W}\mathbf{p}_{C_a}^{C_b} = 0.$$

We can assemble all of these constraints into a single bilateral constraint vector $\mathbf{h}(\mathbf{q}) = 0$, and use almost precisely the recipe above. Note that applying Baumgarte stabilization here would only stabilize in the normal direction, by definition the vector between the two points, which is consistent with having (only) a zero-velocity constraint in the tangential direction. For sliding contacts, we have just the scalar constraint,

$$\mathbf{h}_{A,B}(\mathbf{q}) = {}^{W}\mathbf{p}_{C_{a;z}}^{C_a} - {}^{W}\mathbf{p}_{C_{a;z}}^{C_b} = 0,$$

which says that the distance is zero in the normal direction, plus a constraint the imposes maximum dissipation

$$\lambda_{C_{a;x,y}}^{C_a} = -\mu\lambda_{C_{a;z}}^{C_a} \frac{{}^{W}\mathbf{v}_{C_{a;x,y}}^{C_a}}{|{}^{W}\mathbf{v}_{C_{a;x,y}}^{C_a}|},$$

that can be accommodated when we solve for $\lambda$.

## B.3 The Dynamics of Contact

The dynamics of multibody systems that make and break contact are closely related to the dynamics of constrained systems, but tend to be much more complex. In the simplest form, you can think of non-penetration as an *inequality* constraint: the signed distance between collision bodies must be non-negative. But, as we have seen in the chapters on walking, the transitions when these constraints become active correspond to collisions, and for systems with momentum they require some care. We'll also see that frictional contact adds its own challenges.

There are three main approaches used to modeling contact with "rigid" body systems: 1) rigid contact approximated with stiff compliant contact, 2) hybrid models with collision event detection, impulsive reset maps, and continuous (constrained) dynamics between collision events, and 3) rigid contact approximated with time-averaged forces (impulses) in a time-stepping scheme. Each modeling approach has advantages/disadvantages for different applications.

Before we begin, there is a bit of notation that we will use throughout. Let $\phi(\mathbf{q})$ indicate the relative (signed) distance between two rigid bodies. For rigid contact, we would like to enforce the unilateral constraint:

$$\phi(\mathbf{q}) \geq 0. \tag{19}$$

We'll use $\mathbf{n} = \frac{\partial \phi}{\partial \mathbf{q}}$ to denote the "contact normal", as well as $\mathbf{t}_1$ and $\mathbf{t}_2$ as a basis for the tangents at the contact (orthonormal vectors in the Cartesian space, projected into joint space), all represented as row vectors in joint coordinates.

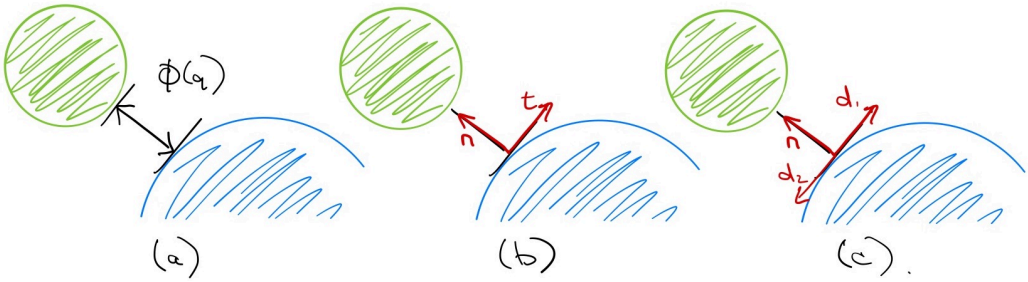



Figure B.2 - Contact coordinates in 2D. (a) The signed distance between contact bodies, $\phi(\mathbf{q})$. (b) The normal ($\mathbf{n}$) and tangent ($\mathbf{t}$) contact vectors -- note that these can be drawn in 2D when the $\mathbf{q}$ is the $x, y$ positions of one of the bodies, but more generally the vectors live in the configuration space. (c) Sometimes it will be helpful for us to express the tangential coordinates using $d_1$ and $d_2$; this will make more sense in the 3D case.

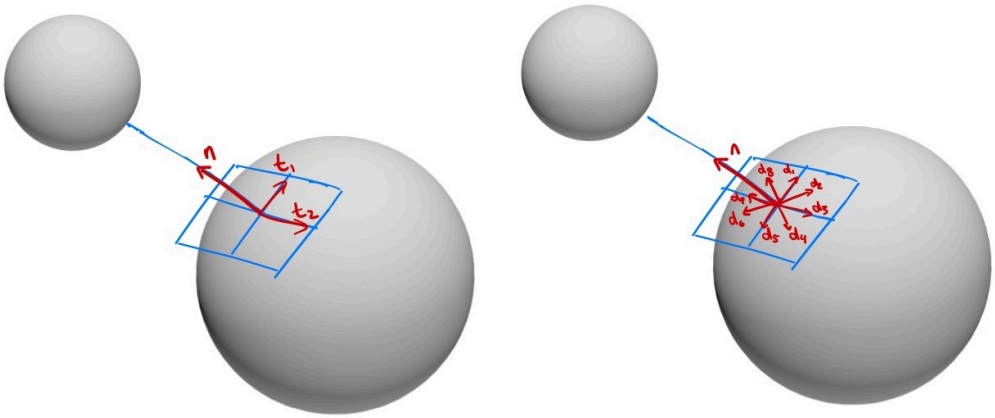



Figure B.3 - Contact coordinates in 3D.

We will also find it useful to assemble the contact normal and tangent vectors into a single matrix, $\mathbf{J}$, that we'll call the *contact Jacobian*:

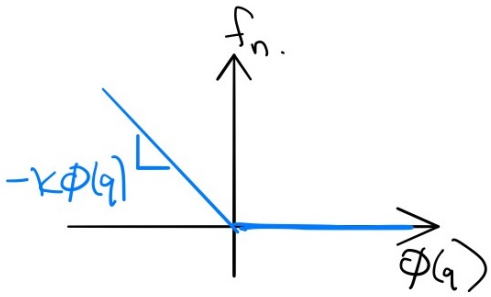
$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} \mathbf{n} \\ \mathbf{t}_1 \\ \mathbf{t}_2 \end{bmatrix}.$$

As written, $\mathbf{Jv}$ gives the relative velocity of the closest points in the contact coordinates; it can be also be extended with three more rows to output a full spatial velocity (e.g. when modeling torsional friction). The generalized force produced by the contact is given by $\mathbf{J}^T\lambda$, where $\lambda = [f_n, f_{t1}, f_{t2}]^T$ :

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} + \mathbf{C}(\mathbf{q}, \mathbf{v})\mathbf{v} = \tau_g(\mathbf{q}) + \mathbf{Bu} + \mathbf{J}^T(\mathbf{q})\lambda. \tag{20}$$

### B.3.1 Compliant Contact Models

Most compliant contact models are conceptually straight-forward: we will implement contact forces using a stiff spring (and damper[11]) that produces forces to resist penetration (and grossly model the dissipation of collision and/or friction). For instance, for the normal force, $f_n$, we can use a simple (piecewise) linear spring law:




Coulomb friction is described by two parameters -- $\mu_{static}$ and $\mu_{dynamic}$ -- which are the coefficients of static and dynamic friction. When the contact tangential velocity (given by $\mathbf{tv}$) is zero, then friction will produce whatever force is necessary to resist motion, up to the threshold $\mu_{static} f_n$. But once this threshold is exceeding, we have slip (non-zero contact tangential velocity); during slip friction will produce a constant drag force $|f_t| = \mu_{dynamic} f_n$ in the direction opposing the motion. This behavior is not a simple function of the current state, but we can approximate it with a continuous function as pictured below.

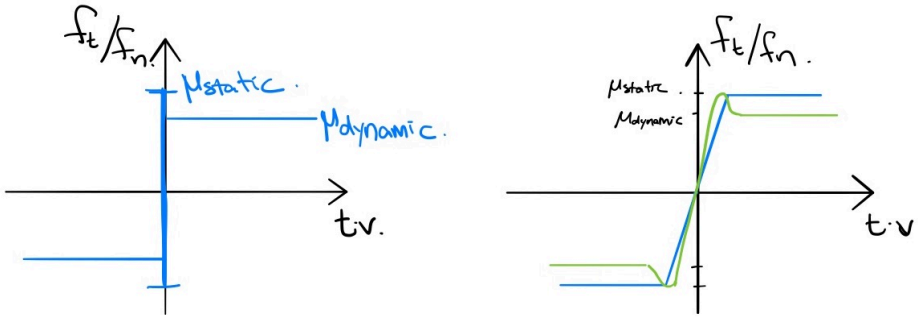



Figure B.5 - (left) The Coloumb friction model. (right) A continuous piecewise-linear approximation of friction (blue) and the Stribeck approximation of Coloumb friction (green); the $x$-axis is the contact tangential velocity, and the $y$-axis is the friction coefficient.

With these two laws, we can recover the contact forces as relatively simple functions of the current state. However, the devil is in the details. There are a few features of this approach that can make it numerically unstable. If you've ever been working in a robotics simulator and watched your robot take a step only to explode out of the ground and go flying through the air, you know what I mean.

In order to tightly approximate the (nearly) rigid contact that most robots make with the world, the stiffness of the contact "springs" must be quite high. For instance, I might want my 180kg humanoid robot model to penetrate into the ground no more than 1mm during steady-state standing. The challenge with adding stiff springs to our model is that this results in stiff differential equations (it is not a coincidence that the word *stiff* is the common term for this in both springs and ODEs). As a result, the best implementations of compliant contact for simulation make use of special-purpose numerical integration recipes (e.g. [12]) and compliant contact models are often difficult to use in e.g. trajectory/feedback optimization.

Note that there is one other type of friction that we talk about in robotics. In joints we also, additionally, model joint damping, often as a linear force proportional to the

velocity which resists motion. Phenomenologically, this is due to a viscous (fluid) drag force produced by lubricants in the joint. The static and dynamic friction described above are sometimes collectively referred to as "dry" friction.

But there is another serious numerical challenge with the basic implementation of this model. Computing the signed-distance function, $\phi(\mathbf{q})$, when it is non-negative is straightforward, but robustly computing the "penetration depth" once two bodies are in collision is not. Consider the case of use $\phi(\mathbf{q})$ to represent the maximum penetration depth of, say, two boxes that are contacting each other. With compliant contact, we must have some penetration to produce any contact force. But the direction of the normal vector, $\mathbf{n} = \frac{\partial \phi}{\partial \mathbf{q}}$, can easily change discontinuously as the point of maximal penetration moves, as I've tried to illustrate here:

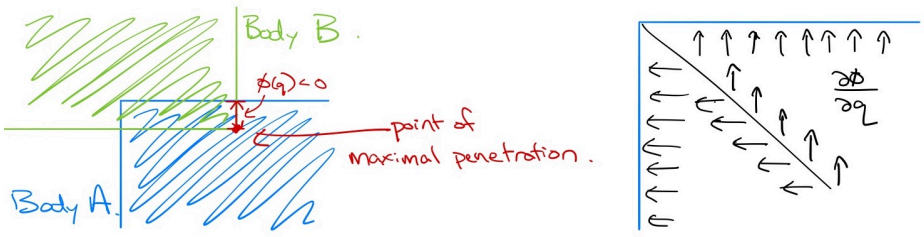



Figure B.6 - (Left) Two boxes in penetration, with the signed distance defined by the maximum penetration depth. (Right) The contact normal for various points of maximal penetration.

If you really think about this example, it actually reveals even more of the foibles of trying to even define the contact points and normals consistently. It seems reasonable to define the point on body B as the point of maximal penetration into body A, but notice that as I've drawn it, that isn't actually unique! The corresponding point on body A should probably be the point on the surface with the minimal distance from the max penetration point (other tempting choices would likely cause the distance to be discontinuous at the onset of penetration). But this whole strategy is asymmetric; why shouldn't I have picked the vector going with maximal penetration into body B? My point is simply that these cases exist, and that even our best software implementations get pretty unsatisfying when you look into the details. And in practice, it's a lot to expect the collision engine to give consistent and smooth contact points out in every situation.

Some of the advantages of this approach include (1) the fact that it is easy to implement, at least for simple geometries, (2) by virtue of being a continuous-time model it can be simulated with error-controlled integrators, and (3) the tightness of the approximation of "rigid" contact can be controlled through relatively intuitive parameters. However, the numerical challenges of dealing with penetration are very real, and they motivate our other two approaches that attempt to more strictly enforce the non-penetration constraints.

### B.3.2 Rigid Contact with Event Detection

### Impulsive Collisions

The collision event is described by the zero-crossings (from positive to negative) of $\phi$. Let us start by assuming frictionless collisions, allowing us to write

$$\mathbf{M}\dot{\mathbf{v}} = \tau + \lambda \mathbf{n}^T, \tag{21}$$

where $\mathbf{n}$ is the "contact normal" we defined above and $\lambda \geq 0$ is now a (scalar) impulsive force that is well-defined when integrated over the time of the collision (denoted $t_c^-$ to $t_c^+$). Integrate both sides of the equation over that (instantaneous) interval:

$$\int_{t_c^-}^{t_c^+} dt \left[\mathbf{M}\dot{\mathbf{v}}\right] = \int_{t_c^-}^{t_c^+} dt \left[\tau + \lambda \mathbf{n}^T\right]$$

Since $\mathbf{M}$, $\tau$, and $\mathbf{n}$ are constants over this interval, we are left with

$$\mathbf{M}\mathbf{v}^+ - \mathbf{M}\mathbf{v}^- = \mathbf{n}^T \int_{t_c^-}^{t_c^+} \lambda dt,$$

where $\mathbf{v}^+$ is short-hand for $\mathbf{v}(t_c^+)$. Multiplying both sides by $\mathbf{n}\mathbf{M}^{-1}$, we have

$$\mathbf{n}\mathbf{v}^+ - \mathbf{n}\mathbf{v}^- = \mathbf{n}\mathbf{M}^{-1}\mathbf{n}^T \int_{t_c^-}^{t_c^+} \lambda dt.$$

After the collision, we have $\dot{\phi}^+ = -e\dot{\phi}^-$, with $0 \leq e \leq 1$ denoting the coefficient of restitution, yielding:

$$\mathbf{n}\mathbf{v}^+ - \mathbf{n}\mathbf{v}^- = -(1+e)\mathbf{n}\mathbf{v}^-,$$

and therefore

$$\int_{t_c^-}^{t_c^+} \lambda dt = -(1+e)\left[\mathbf{n}\mathbf{M}^{-1}\mathbf{n}^T\right]^{\#}\mathbf{n}\mathbf{v}^-.$$

I've used the notation $A^{\#}$ here to denote the pseudoinverse of $A$ (I would normally write $A^+$, but have changed it for this section to avoid confusion). Substituting this back in above results in

$$\mathbf{v}^+ = \left[\mathbf{I} - (1+e)\mathbf{M}^{-1}\mathbf{n}^T\left[\mathbf{n}\mathbf{M}^{-1}\mathbf{n}^T\right]^{\#}\mathbf{n}\right]\mathbf{v}^-. \tag{22}$$

We can add friction at the contact. Rigid bodies will almost always experience contact at only a point, so we typically ignore torsional friction [7], and model only tangential friction by extending $\mathbf{n}$ to a matrix

$$\mathbf{J} = \begin{bmatrix} \mathbf{n} \\ \mathbf{t}_1 \\ \mathbf{t}_2 \end{bmatrix},$$

to capture the gradient of the contact location tangent to the contact surface. Then $\lambda$ becomes a Cartesian vector representing the contact impulse, and (for infinite friction) the post-impact velocity condition becomes $\mathbf{J}\mathbf{v}^+ = \text{diag}(-e, 0, 0)\mathbf{J}\mathbf{v}^-$, resulting in the equations:

$$\mathbf{v}^+ = \left[\mathbf{I} - \mathbf{M}^{-1}\mathbf{J}^T \text{diag}(1+e, 1, 1)\left[\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^T\right]^{\#}\mathbf{J}\right]\mathbf{v}^-. \tag{23}$$

Often in optimization, we can directly use the implicit form

$$\mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = \mathbf{J}^T\mathbf{\Lambda}$$
$$\mathbf{J}\mathbf{v}^+ = \text{diag}(-e, 0, 0)\mathbf{J}\mathbf{v}^-$$

If $\mathbf{\Lambda}$ is restricted to a friction cone, as in Coulomb friction, in general we have to solve an optimization to solve for $\mathbf{v}^+$ subject to the friction cone constraints on $\mathbf{\Lambda}$.

**Example B.4 (A spinning ball bouncing on the ground.)**

Imagine a ball (a hollow-sphere) in the plane with mass $m$, radius $r$. The configuration is given by $q = [x, z, \theta]^T$. The equations of motion are

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & \frac{2}{3}mr^2 \end{bmatrix}\ddot{\mathbf{q}} = \begin{bmatrix} 0 \\ -g \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & r \end{bmatrix}\begin{bmatrix} \lambda_z \\ \lambda_x \end{bmatrix} = \tau_g + \mathbf{J}^T\lambda,$$

where $\lambda$ are the contact forces (which are zero except during the impulsive collision). Given a coefficient of restitution $e$ and a collision with a horizontal ground, the post-impact velocities are:

$$\dot{\mathbf{q}}^+ = \begin{bmatrix} \frac{3}{5} & 0 & -\frac{2}{5}r \\ 0 & -e & 0 \\ -\frac{3}{5r} & 0 & \frac{2}{5} \end{bmatrix}\dot{\mathbf{q}}^-.$$

**Putting it all together**

We can put the bilateral constraint equations and the impulsive collision equations together to implement a hybrid model for unilateral constraints of the form. Let us generalize

$$\phi(\mathbf{q}) \geq 0, \tag{24}$$

to be the vector of all pairwise (signed) distance functions between rigid bodies; this vector describes all possible contacts. At every moment, some subset of the contacts are active, and can be treated as a bilateral constraint ($\phi_i = 0$). The guards of the hybrid model are when an inactive constraint becomes active ($\phi_i > 0 \rightarrow \phi_i = 0$), or when an active constraint becomes inactive ($\lambda_i > 0 \rightarrow \lambda_i = 0$ and $\dot{\phi}_i > 0$). Note that collision events will (almost always) only result in new active constraints when $e = 0$, e.g. we have pure inelastic collisions, because elastic collisions will rarely permit sustained contact.

If the contact involves Coulomb friction, then the transitions between sticking and sliding can be modeled as additional hybrid guards.

### B.3.3 Time-stepping Approximations for Rigid Contact

So far we have seen two different approaches to enforcing the inequality constraints of contact, $\phi(\mathbf{q}) \geq 0$ and the friction cone. First we introduced compliant contact which effectively enforces non-penetration with a stiff spring. Then we discussed event detection as a means to switch between different models which treat the active constraints as equality constraints. But, perhaps surprisingly, one of the most popular and scalable approaches is something different: it involves formulating a mathematical program that can solve the inequality constraints directly on each time step of the simulation. These algorithms may be more expensive to compute on each time step, but they allow for stable simulations with potentially much larger and more consistent time steps.

#### Complementarity formulations

What class of mathematical program due we need to simulate contact? The discrete nature of contact suggests that we might need some form of combinatorial optimization. Indeed, the most common transcription is to a Linear Complementarity Problem (LCP) [13], as introduced by [14] and [15]. An LCP is typically written as

$$\begin{aligned} \text{find}_{\mathbf{w},\mathbf{z}} \quad \text{subject to} \quad & \mathbf{w} = \mathbf{q} + \mathbf{M}\mathbf{z}, \\ & \mathbf{w} \geq 0, \mathbf{z} \geq 0, \mathbf{w}^T\mathbf{z} = 0. \end{aligned}$$

They are directly related to the optimality conditions of a (potentially non-convex) quadratic program and [15]> showed that the LCP's generated by our rigid-body contact problems can be solved by Lemke's algorithm. As short-hand we will write these complementarity constraints as

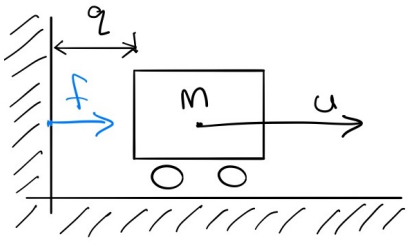
$$\text{find}_{\mathbf{z}} \quad \text{subject to} \quad 0 \leq (\mathbf{q} + \mathbf{M}\mathbf{z}) \perp \mathbf{z} \geq 0.$$

Rather than dive into the full transcription, which has many terms and can be relatively difficult to parse, let me start with two very simple examples.

**Example B.5 (Time-stepping LCP: Normal force)**

Consider our favorite simple mass being actuated by a horizontal force (with the double integrator dynamics), but this time we will add a wall that will prevent the cart position from taking negative values: our non-penetration constraint is $q \geq 0$. Physically, this constraint is implemented by a normal (horizontal) force, $f$, yielding the equations of motion:

$$m\ddot{q} = u + f.$$




$f$ is defined as the force required to enforce the non-penetration constraint; certainly the following are true: $f \geq 0$ and $q \cdot f = 0$. $q \cdot f = 0$ is the "complementarity constraint", and you can read it here as "either q is zero or force is zero" (or both); it is our "no force at a distance" constraint, and it is clearly non-convex. It turns out that satisfying these constraints, plus $q \geq 0$, is sufficient to fully define $f$.

To define the LCP, we first discretize time, by approximating

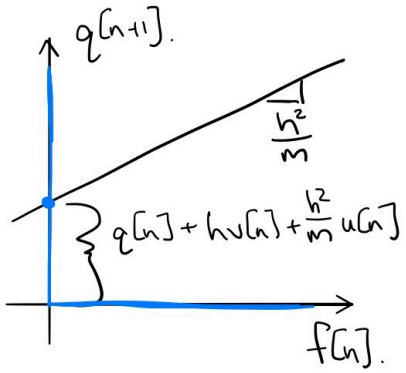
$$q[n+1] = q[n] + hv[n+1],$$
$$v[n+1] = v[n] + \frac{h}{m}(u[n] + f[n]).$$

This is almost the standard Euler approximation, but note the use of $v[n+1]$ in the right-hand side of the first equation -- this is actually a semi-implicit Euler approximation, and this choice is essential in the derivation.

Given $h, q[n], v[n]$, and $u[n]$, we can solve for $f[n]$ and $q[n+1]$ simultaneously, by solving the following LCP:

$$q[n+1] = \left[q[n] + hv[n] + \frac{h^2}{m}u[n]\right] + \frac{h^2}{m}f[n]$$
$$q[n+1] \geq 0, \quad f[n] \geq 0, \quad q[n+1] \cdot f[n] = 0.$$

Take a moment and convince yourself that this fits into the LCP prescription given above.




Note: Please don't confuse this visualization with the more common visualization of the solution space for an LCP (in two or more dimensions) in terms of "complementary cones"[13].

Perhaps it's also helpful to plot the solution, $q[n+1], f[n]$ as a function of $q[n], v[n]$. I've done it here for $m = 1, h = 0.1, u[n] = 0$:

In the (time-stepping, "impulse-velocity") LCP formulation, we write the contact problem in it's combinatorial form. In the simple example above, the complementarity constraints force any solution to lie on *either* the positive x-axis ( $f[n] \geq 0$) *or* the positive y-axis ($q[n+1] \geq 0$). The equality constraint is simply a line that will intersect with this constraint manifold at the solution. But in this frictionless case, it is important to realize that these are simply the optimality conditions of a convex optimization problem: the discrete-time version of Gauss's principle that we used above. Using $\mathbf{v}'$ as shorthand for $\mathbf{v}[n+1]$, and replacing $\ddot{\mathbf{q}} = \frac{\mathbf{v}'-\mathbf{v}}{h}$ in Eq (13) and scaling the objective by $h^2$ we have:

$$\begin{aligned} \min_{\mathbf{v}'} \quad & \frac{1}{2}\left(\mathbf{v}' - \mathbf{v} - h\mathbf{M}^{-1}\tau\right)^T \mathbf{M}\left(\mathbf{v}' - \mathbf{v} - h\mathbf{M}^{-1}\tau\right) \\ \text{subject to} \quad & \frac{1}{h}\phi(\mathbf{q}') = \frac{1}{h}\phi(\mathbf{q} + h\mathbf{v}') \approx \frac{1}{h}\phi(\mathbf{q}) + \mathbf{n}\mathbf{v}' \geq 0. \end{aligned}$$

The objective is even cleaner/more intuitive if we denote the next step velocity that would have occurred before the contact impulse is applied as $\mathbf{v}^- = \mathbf{v} + h\mathbf{M}^{-1}\tau$:

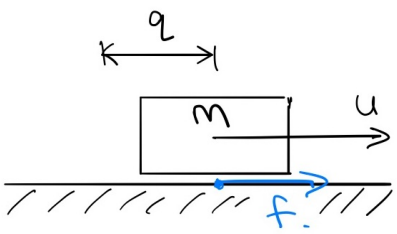
$$\begin{aligned} \min_{\mathbf{v}'} \quad & \frac{1}{2}\left(\mathbf{v}' - \mathbf{v}^-\right)^T \mathbf{M}\left(\mathbf{v}' - \mathbf{v}^-\right) \\ \text{subject to} \quad & \frac{1}{h}\phi(\mathbf{q}') \geq 0. \end{aligned}$$

The LCP for the frictionless contact dynamics is precisely the optimality conditions of this convex (because $\mathbf{M} \succeq 0$) quadratic program, and once again the contact impulse, $\mathbf{f} \geq 0$, plays the role of the Lagrange multiplier (with units $N \cdot s$).

So why do we talk so much about LCPs instead of QPs? Well LCPs can also represent a wider class of problems, which is what we arrive at with the standard transcription of Coulomb friction. In the limit of infinite friction, then we could add an additional constraint that the tangential velocity at each contact point was equal to zero (but these equation may not always have a feasible solution). Once we admit limits on the magnitude of the friction forces, the non-convexity of the disjunctive form rear's its ugly head.

**Example B.6 (Time-stepping LCP: Coulomb Friction)**

We can use LCP to find a feasible solution with Coulomb friction, but it requires some gymnastics with slack variables to make it work. For this case in particular, I believe a very simple example is best. Let's take our brick and remove the wall and the wheels (so we now have friction with the ground).




The dynamics are the same as our previous example,

$$m\ddot{q} = u + f,$$

but this time I'm using $f$ for the friction force which is inside the friction cone if $\dot{q} = 0$ and on the boundary of the friction cone if $\dot{q} \neq 0$; this is known as the principle of maximum dissipation[14]. Here the magnitude of the normal force is always $mg$, so we have $|f| \leq \mu mg$, where $\mu$ is the coefficient of friction. And we will use the same semi-implicit Euler approximation to cast this into discrete time.

Now, to write the friction constraints as an LCP, we must introduce some slack variables. First, we'll break the force into a positive component and a negative component: $f[n] = f^+ - f^-$. And we will introduce one more variable $v_{abs}$ which will be non-zero if the velocity is non-zero (in either direction). Now we can write the LCP:

$$\begin{aligned} &\text{find}_{v_{abs}, f^+, f^-} \quad \text{subject to} \\ &0 \leq v_{abs} + v[n+1] \quad \perp \quad f^+ \geq 0, \\ &0 \leq v_{abs} - v[n+1] \quad \perp \quad f^- \geq 0, \\ &0 \leq \mu mg - f^+ - f^- \quad \perp \quad v_{abs} \geq 0, \end{aligned}$$

where each instance of $v[n+1]$ we write out with

$$v[n+1] = v[n] + \frac{h}{m}(u + f^+ - f^-).$$

It's enough to make your head spin! But if you work it out, all of the constraints we want are there. For example, for $v[n+1] > 0$, then we must have $f^+ = 0$, $v_{abs} = v[n+1]$, and $f^- = \mu mg$. When $v[n+1] = 0$, we can have $v_{abs} = 0$, $f^+ - f^- \leq \mu mg$, and those forces must add up to make $v[n+1] = 0$.

We can put it all together and write an LCP with both normal forces and friction forces, related by Coulomb friction (using a polyhedral approximation of the friction cone in 3d)[14]. Although the solution to any LCP can also be represented as the solution to a QP, the QP for this problem is non-convex.

### Anitescu's convex formulation

In [16], Anitescu described the natural convex formulation of this problem by dropping the maximum dissipation inequalities and allowing any force inside the friction cone. Consider the following optimization:

$$\begin{aligned} \min_{\mathbf{v}'} \quad & \frac{1}{2}\left(\mathbf{v}' - \mathbf{v}^-\right)^T \mathbf{M} \left(\mathbf{v}' - \mathbf{v}^-\right) \\ \text{subject to} \quad & \frac{1}{h}\phi(\mathbf{q}') + \mu \mathbf{d}_i \mathbf{v}' \geq 0, \qquad \forall i \in 1, \ldots, m \end{aligned}$$

where $\mathbf{d}_i, \forall i \in 1, \ldots, m$ are a set of tangent row vectors in joint coordinates parameterizing the polyhedral approximation of the friction cone (as in [14]). Importantly, for each $\mathbf{d}_i$ we must also have $-\mathbf{d}_i$ in the set. By writing the Lagrangian,

$$L(\mathbf{v}', \beta) = \frac{1}{2}(\mathbf{v}' - \mathbf{v}^-)^T\mathbf{M}(\mathbf{v}' - \mathbf{v}^-) - \sum_i \beta_i \left(\frac{1}{h}\phi(\mathbf{q}) + (\mathbf{n} + \mu\mathbf{d}_i)\mathbf{v}'\right),$$

and checking the stationarity condition:

$$\frac{\partial L}{\partial \mathbf{v}'}^T = \mathbf{M}(\mathbf{v}' - \mathbf{v}) - h\tau - \sum_i \beta_i(\mathbf{n} + \mu\mathbf{d}_i)^T = 0,$$

we can see that the Lagrange multiplier, $\beta_i \geq 0$, associated with the $i$th constraint is the magnitude of the impulse (with units $N \cdot s$) in the direction $\mathbf{n} - \mu\mathbf{d}_i$, where $\mathbf{n} = \frac{\partial \phi}{\partial \mathbf{q}}$ is the contact normal; the sum of the forces is an affine combination of these extreme rays of the polyhedral friction cone. As written, the optimization is a QP.

But be careful! Although the primal solution was convex, and the dual problems are always convex, the objective here can be positive semi-definite. This isn't a mistake -- [16] describes a few simple examples where the solution to $\mathbf{v}'$ is unique, but the impulses that produce it are not (think of a table with four legs).

When the tangential velocity is zero, this constraint is tight; for sliding contact the relaxation effectively causes the contact to "hydroplane" up out of contact, because $\phi(\mathbf{q}') \geq h\mu\mathbf{d}_i\mathbf{v}'$. It seems like a quite reasonable approximation to me, especially for small $h$!

Let's continue on and write the dual program. To make the notation a little better, let us stack the Jacobian vectors into $\mathbf{J}_\beta$, such that the $i$th row is $\mathbf{n} + \mu\mathbf{d_i}$, so that we have

$$\frac{\partial L}{\partial \mathbf{v}'}^T = \mathbf{M}(\mathbf{v}' - \mathbf{v}) - h\tau - \mathbf{J}_\beta^T\beta = 0.$$

Substituting this back into the Lagrangian give us the dual program:

$$\min_{\beta \geq 0} \frac{1}{2}\beta^T\mathbf{J}_\beta\mathbf{M}^{-1}\mathbf{J}_\beta^T\beta + \frac{1}{h}\phi(\mathbf{q})\sum_i \beta_i + \beta^T\mathbf{J}_\beta\mathbf{v}^-.$$

One final note: I've written just one contact point here to simplify the notation, but of course we repeat the constraint(s) for each contact point; [16] studies the typical case of only including potential contact pairs with $\phi(\mathbf{q}) \leq \epsilon$, for small $\epsilon \geq 0$.

### Todorov's regularization

According to [17], the popular MuJoCo simulator uses a slight variation of this convex relaxation, with the optimization:

$$\begin{aligned} \min_\lambda \quad & \frac{1}{2}\lambda^T\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^T\lambda + \frac{1}{h}\lambda^T\left(\mathbf{J}\mathbf{v}^- - \dot{\mathbf{x}}^d\right), \\ \text{subject to} \quad & \lambda \in \mathcal{FC}(\mathbf{q}), \end{aligned}$$

where $\dot{\mathbf{x}}^d$ is a "desired contact velocity", and $\mathcal{FC}(\mathbf{q})$ describes the friction cone. The friction cone constraint looks new but is not; observe that $\mathbf{J}^T\lambda = \mathbf{J}_\beta^T\beta$, where $\beta \geq 0$ is a clever parameterization of the friction cone in the polyhedral case. The bigger difference is in the linear term: [17] proposed $\dot{\mathbf{x}}^d = \mathbf{J}\mathbf{v} - h\mathcal{B}\mathbf{J}\mathbf{v} - h\mathcal{K}[\phi(\mathbf{q}), 0, 0]^T$, with $\mathcal{B}$ and $\mathcal{K}$ stabilization gains that are chosen to yield a critically damped response.

How should we interpret this? If you expand the linear terms, you will see that this is almost exactly the dual form we get from the position equality constraints formulation, including the Baumgarte stabilization. It's interesting to think about the implications of this formulation -- like the Anitescu relaxation, it is possible to get some "force at a distance" because we have not in any way expressed that $\lambda = 0$ when $\phi(\mathbf{q}) > 0$. In Anitescu, it happened in a velocity-dependent boundary layer; here it could happen if you are "coming in hot" -- moving towards contact with enough relative velocity that the stabilization terms want to slow you down.

In dual form, it is natural to consider the full conic description of the friction cone:

$$\mathcal{FC}(\mathbf{q}) = \left\{ \lambda = [f_n, f_{t1}, f_{t2}]^T \middle| f_n \geq 0, \sqrt{f_{t1}^2 + f_{t2}^2} \leq \mu f_n \right\}.$$

The resulting dual optimization is has a quadratic objective and second-order cone constraints (SOCP).

There are a number of other important relaxations that happen in MuJoCo. To address the positive indefiniteness in the objective, [17] relaxes the dual objective by adding a small term to the diagonal. This guarantees convexity, but the convex optimization can still be poorly conditioned. The stabilization terms in the objective are another form of relaxation, and [17] also implements adding additional stabilization to the inertial matrix, called the "implicit damping inertia". These relaxations can dramatically improve simulation speed, both by making each convex optimization faster to solve, and by allowing the simulator to take larger integration time steps. MuJoCo boasts a special-purpose solver that can simulate complex scenes at seriously impressive speeds -- often orders of magnitude faster than real time. But these relaxations can also be abused -- it's unfortunately quite common to see physically absurd MuJoCo simulations, especially when researchers who don't care much about the physics start changing simulation parameters to optimize their learning curves!

### The Semi-Analytic Primal (SAP) solver

The contact solver that we use in Drake further extends and improves upon this line of work... [18].

### Beyond Point Contact

Coming soon... Box-on-plane example. Multi-contact.

Also a single point force cannot capture effects like torsional friction, and performs badly in some very simple cases (imaging a box sitting on a table). As a result, many of the most effective algorithms for contact restrict themselves to very limited/simplified

geometry. For instance, one can place "point contacts" (zero-radius spheres) in the four corners of a robot's foot; in this case adding forces at exactly those four points as they penetrate some other body/mesh can give more consistent contact force locations/directions. A surprising number of simulators, especially for legged robots, do this.

In practice, most collision detection algorithms will return a list of "collision point pairs" given the list of bodies (with one point pair per pair of bodies in collision, or more if they are using the aforementioned "multi-contact" heuristics), and our contact force model simply attaches springs between these pairs. Given a point-pair, $p_A$ and $p_B$, both in world coordinates, ...

Hydroelastic model in drake[19].

## B.4 Variational mechanics

The field of analytic mechanics (or variational mechanics), has given us a number of different and potentially powerful interpretations of these equations. While our principle of virtual work was very much rooted in mechanical work, these variational principles have proven to be much more general. In the context of these notes, the variational principles also provide some deep connections between mechanics and optimization. So let's consider some of the equations above, but through the more general lens of variational mechanics.

### B.4.1 Virtual work

Most of us are comfortable with the idea that a body is in equilibrium if the sum of all forces acting on that body (including gravity) is zero:

$$\sum_i \mathbf{f}_i = 0.$$

It turns out that there are some limitations in working with forces directly: forces require us to (arbitrarily) define a coordinate system, and to achieve force balance we might need to consider *all* of the forces in the system, including internal forces like the forces at the joints that hold our robots together.

So it turns out to be better to think not about forces directly, but to think instead about the *work*, $w$, done by those forces. Work is force times distance. It's a scalar quantity, and it is invariant to the choice of coordinate system. Even for a system that is in equilibrium, we can reason about the *virtual work* that would occur if we were to make an infinitesimal change to the configuration of the system. Even better, since we know that the internal forces do no work, we can ignore them completely and only consider the virtual work that occurs due to a variation in the configuration variables. A multibody system in equilibrium must have zero virtual work:

$$\delta w = \sum_i \tau_i^T \delta \mathbf{q} = 0.$$

Notice that these forces are the generalized forces (they have the same dimension as $\mathbf{q}$). We can map between a Cartesian force and the generalized force using a kinematic

Jacobian: $\tau_i = \mathbf{J}_i^T(\mathbf{q})\mathbf{f}_i$. In particular, forces due to gravity, or any forces described by a "potential energy" function, are can be written as

$$\tau_g = -\frac{\partial U_g(\mathbf{q})}{\partial \mathbf{q}}^T.$$

If all of the generalized forces can be derived from such a potential function, then we can go even further. For a system to be in equilibrium, we must have that varations in the potential energy function must be zero, $\delta U(\mathbf{q}) = 0$. This follows simply from $\delta U = \frac{\partial U}{\partial \mathbf{q}}\delta\mathbf{q}$ and the principle of virtual work. Moreover, for the system to be in a *stable* equilibrium, $\mathbf{q}$ must not only be a stationary point of the potential, but a *minimum* of the potential. This is our first important connection in this section between mechanics and optimization.

### B.4.2 D'Alembert's principle and the force of inertia

What about bodies in motion? d'Alembert's principle states that we can reduce dynamics problems to statics problems by simply including a "force of inertia", which is the familiar mass times acceleration for simple particles. In order to have $\delta w = \sum_i \tau_i^T \delta\mathbf{q} = 0$, for all $\delta\mathbf{q}$, we must have $\sum_i \tau_i = 0$, so for multibody systems we know that the force of inertia must be:

$$\tau_{inertia} = -\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}.$$

But where did that term come from? Is it the result of nature "optimizing" some scalar function?

A natural candidate would be the kinetic energy: $T(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{M}(\mathbf{q})\dot{\mathbf{q}}$. What would it mean to take a variation of the kinetic energy, $\delta T$? So far we've only had variations with respect to $\delta\mathbf{q}$, but the kinetic energy depends on both $\mathbf{q}$ and $\dot{\mathbf{q}}$. We cannot vary these variables independently, but how are they related?

The solution is to take a variation not at a single point, but along a trajectory, where the relationship between $\delta\mathbf{q}$ and $\delta\dot{\mathbf{q}}$ is well defined. If we consider

$$A(\mathbf{q}(t)) = \int_{t_0}^{t_1} T(\mathbf{q}, \dot{\mathbf{q}})dt,$$

then a variation of $A$ (using the calculus of variations) w.r.t. any trajectory $\mathbf{q}(t)$, will take the form:

$$\delta A = A(\mathbf{q}(t) + \delta\mathbf{q}(t)) - A(\mathbf{q}(t) = \int_{t_0}^{t_1} T(\mathbf{q} + \delta\mathbf{q}(t), \dot{\mathbf{q}} + \delta\dot{\mathbf{q}}(t))dt - \int_{t_0}^{t_1} T(\mathbf{q}(t), \dot{\mathbf{q}}(t$$

where $\delta\mathbf{q}(t)$ is defined to be an arbitrary (small) differentiable function in the space of $\mathbf{q}$ that is zero at $t_0$ and $t_1$.

▶ The derivation takes a few steps, which can be found in most mechanics texts. Click here to see my version.

The result is that we find that the natural generalization of $-\frac{\partial U(\mathbf{q})}{\partial \mathbf{q}}^T$ for stationary points of variations of quantities that also depend on velocity is

$$\tau_{inertia} = -\frac{\partial T}{\partial \mathbf{q}}^T + \frac{d}{dt}\frac{\partial T}{\partial \dot{\mathbf{q}}}^T.$$

This is exactly (the negation of) our familiar $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$.

### B.4.3 Principle of Stationary Action

The integral quantity $A$ that we used in the last section is commonly called the "action" of the system. More generally, the action is defined as

$$A = \int_{t_0}^{t_1} L(\mathbf{q}, \dot{\mathbf{q}}) dt,$$

where $L(\mathbf{q}, \dot{\mathbf{q}})$ is an appropriate "Lagrangian" for the system. The Lagrangian has units of energy, but it is not uniquely defined. Any function which generates the correct equations of motion, in agreement with physical laws, can be taken as a Lagrangian. For our mechanical systems, the familiar form of the Lagrangian is what we presented above: $L = T - U$ (kinetic energy minus potential energy). Since the terms involving $U(\mathbf{q})$ do not depend directly on $\dot{\mathbf{q}}$, they only contribute the $\frac{\partial U}{\partial \mathbf{q}}^T$ terms that we expect, and taking $\delta A = 0$ for this Lagrangian is a way to generate the entire equations of motion in one go. For our purposes, where we care about conservative forces but also control inputs (as generalized forces $\mathbf{Bu}$), the principle of virtual work tells us that we will have $\delta A + \int_{t_0}^{t_1} \mathbf{u}^T \mathbf{B}^T \delta\mathbf{q}(t) dt = 0$.

The viewpoint of variational mechanics is that it is this scalar quantity $L(\mathbf{q}, \dot{\mathbf{q}})$ that should be the central object of study (as opposed to coordinate dependent forces), and this has proven to generalize beautifully. [20] says

> The principle of least action -- really the principle of stationary action -- is the most compact form of the classical laws of physics. This simple rule (it can be written in a single line) summarizes everything! Not only the principles of classical mechanics, but electromagnetism, general relativity, quantum mechanics, everything known about chemistry -- right down to the ultimate known constituents of matter, elementary particles.

### B.4.4 Hamiltonian Mechanics

In some cases, it might be preferable to use the Hamiltonian formulation of the dynamics, with state variables $\mathbf{q}$ and $\mathbf{p}$ (instead of $\dot{\mathbf{q}}$), where $\mathbf{p} = \frac{\partial L}{\partial \dot{\mathbf{q}}}^T = \mathbf{M}\dot{\mathbf{q}}$ results in the dynamics:

$$\begin{aligned}\mathbf{M}(\mathbf{q})\dot{\mathbf{q}} =& \mathbf{p} \\ \dot{\mathbf{p}} =& \mathbf{c}_H(\mathbf{q}, \mathbf{p}) + \tau_g(\mathbf{q}) + \mathbf{Bu}, \qquad \text{where} \quad \mathbf{c}_H(\mathbf{q}, \mathbf{p}) = \frac{1}{2}\left(\frac{\partial \left[\dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}\right]}{\partial \mathbf{q}}\right)^T.\end{aligned}$$

Recall that the Hamiltonian is $H(\mathbf{p}, \mathbf{q}) = \mathbf{p}^T \dot{\mathbf{q}} - L(\mathbf{q}, \dot{\mathbf{q}})$; these equations of motion are the familiar $\dot{\mathbf{q}} = \frac{\partial H}{\partial \mathbf{p}}^T, \dot{\mathbf{p}} = -\frac{\partial H}{\partial \mathbf{q}}^T$.

## B.5 Exercises

### Exercise B.1 (Time-stepping LCP dynamics for block collision)

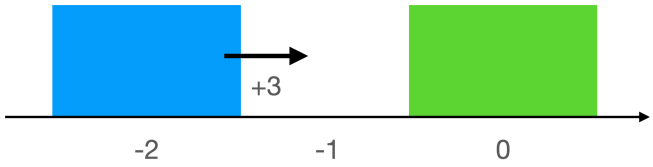



Figure B.11 - Initial conditions for block collision scenario.

Suppose you have a block A of width 1, block B of width 1, interacting on a friction-less surface. The state of a single block in our system is $\vec{x} = \begin{bmatrix} q_x \\ \dot{q}_x \end{bmatrix}$, which represents the horizontal position (where the position point is located at the geometric center of the block) and horizontal velocity of the associated block. We have as our initial condition $\vec{x}_0^{(A)} = \begin{bmatrix} -2 \\ 3 \end{bmatrix}$ and $\vec{x}_0^{(B)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$. In other words, at time step $0$ block A is moving towards a stationary block B located at the origin.

a. Derive the positions, velocities, and contact forces at time steps 1, 2, and 3 for the two block collision scenario described at the start of this problem, under implicit Euler integration. Use time step $h = 1.0$ and block masses $m_1 = m_2 = 1$.
   *Hint:* See Example B.4 in the Appendix setting up the LCP optimization problem for a single cart and wall under semi-implicit integration. In implicit Euler integration, both the position and velocity are integrated with backward finite differences ($x[n+1] = x[n] + hf[n+1]$), whereas in the semi-implicit integration, only the position is integrated with backward finite differences.
   *Hint:* To solve your LCP by hand, you should use the fact that time-stepping LCP produces perfectly inelastic collisions. What does the perfectly inelastic nature of collision imply about the signed distance during collision?
b. Since the block B is stationary, we might expect $q_{1,x}^{(A)} = -1$. Why is this not the case?
c. What would you expect the velocities to be post-collision for perfectly inelastic collision and under conservation of momentum in continuous-time physics? How do these velocities compare with your time-stepped velocities?

## References

1. John Craig, "Introduction to Robotics: Mechanics and Control", Addison Wesley , January, 1989.

2. Cornelius Lanczos, "The variational principles of mechanics", University of Toronto Press , no. no. 4, 1970.

3. H. Asada and J.E. Slotine, "Robot Analysis and Control", , pp. 93-131, 1986.

4. Roy Featherstone, "Efficient Factorization of the Joint Space Inertia Matrix for Branched Kinematic Trees", *International Journal of Robotics Research*, vol. 24, no. 6, pp. 487â€“500, 2005.

5. Jean-Jacques E. Slotine and Weiping Li, "Applied Nonlinear Control", Prentice Hall , October, 1990.

6. Vincent Duindam, "Port-Based Modeling and Control for Efficient Bipedal Walking Robots", PhD thesis, University of Twente, March, 2006.

7. Roy Featherstone, "Rigid Body Dynamics Algorithms", Springer , 2007.

8. Brian Mirtich, "Impulse-based Dynamic Simulation of Rigid Body Systems", PhD thesis, University of California at Berkeley, 1996.

9. Abhinandan Jain, "Robot and Multibody Dynamics: Analysis and Algorithms", Springer US , 2011.

10. Firdaus E Udwadia and Robert E Kalaba, "A new perspective on constrained motion", *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences*, vol. 439, no. 1906, pp. 407--410, 1992.

11. K. H. Hunt and F. R. E. Crossley, "Coefficient of restitution interpreted as damping in vibroimpact", *Journal of Applied Mechanics*, vol. 42 Series E, no. 2, pp. 440-445, 1975.

12. Alejandro M Castro and Ante Qu and Naveen Kuppuswamy and Alex Alspach and Michael Sherman, "A Transition-Aware Method for the Simulation of Compliant Contact with Regularized Friction", *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1859--1866, 2020.

13. Katta G Murty and Feng-Tien Yu, "Linear complementarity, linear and nonlinear programming", Citeseer , vol. 3, 1988.

14. D.E. Stewart and J.C. Trinkle, "AN IMPLICIT TIME-STEPPING SCHEME FOR RIGID BODY DYNAMICS WITH INELASTIC COLLISIONS AND COULOMB FRICTION", *International Journal for Numerical Methods in Engineering*, vol. 39, no. 15, pp. 2673--2691, 1996.

15. M. Anitescu and F.A. Potra, "Formulating dynamic multi-rigid-body contact problems with friction as solvable linear complementarity problems", *Nonlinear Dynamics*, vol. 14, no. 3, pp. 231--247, 1997.

16. Mihai Anitescu, "Optimization-based simulation of nonsmooth rigid multibody dynamics", *Mathematical Programming*, vol. 105, no. 1, pp. 113--143, jan, 2006.

17. Emanuel Todorov, "Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in MuJoCo", *2014 IEEE International Conference on Robotics and Automation (ICRA)* , pp. 6054--6061, 2014.

18. Alejandro Castro and Frank Permenter and Xuchen Han, "An Unconstrained Convex Formulation of Compliant Contact", *arXiv preprint arXiv:2110.10107*, 2021.

19. Ryan Elandt and Evan Drumwright and Michael Sherman and Andy Ruina, "A pressure field model for fast, robust approximation of net contact force and moment between nominally rigid objects", , pp. 8238-8245, 11, 2019.

20. Leonard Susskind and George Hrabovsky, "The theoretical minimum: what you need to know to start doing physics", Basic Books , 2014.

Previous Chapter | Table of contents | Next Chapter

Accessibility